

Spiegazione della scelta delle funzioni

ML per riconoscimento caratteri (lettere e numeri)

Creazione del modello e prima correzione

Inizialmente, il Machine Learning per riconoscere lettere scritte a mano fu scritto con una MLP. Non funzionava bene però, abbiamo quindi deciso di crearne un'altra con l'architettura MLP.

La differenza tra queste due è come vengono elaborati i dati in ingresso:

- nelle MLP viene richiesto un vettore piatto di dati (che nel nostro caso era l'appiattimento di una foto 28×28 in un vettore da x784), che fa perdere la percezione spaziale e geometrica delle lettere.
- nelle CNN invece le immagini vengono analizzate come tali, come una griglia 2D.

Per passare dallo script fatto in MLP a quello fatto in CNN bastano due semplici modifiche:

- applicare la normalizzazione tra 0 e 1 (per il colore) e ridare alle immagini la dimensione 2D, codice:

```
# Normalizzazione tra 0 e 1 (Già presente nel tuo codice)
X_train = X_train / 255.0
X_test  = X_test  / 255.0

# NUOVO: Reshape dei dati per renderli compatibili con i layer Conv2D
X_train = X_train.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
X_test  = X_test.reshape(-1, IMG_SIZE, IMG_SIZE, 1)
```

- sostituzione del blocco del modello MLP con l'architettura convoluzionale, codice:

```
# Nuovo Modello CNN
model = keras.Sequential()

# Layer di Input: riceve immagini 28x28 con 1 canale
model.add(layers.Input(shape=(IMG_SIZE, IMG_SIZE, 1)))

# Primo blocco Convoluzionale (estrae le prime feature come b
ordi e linee)
model.add(layers.Conv2D(32, kernel_size=(3, 3), activation="r
elu"))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))

# Secondo blocco Convoluzionale (estrae feature più complesse
come curve e angoli)
model.add(layers.Conv2D(64, kernel_size=(3, 3), activation="r
elu"))
model.add(layers.MaxPooling2D(pool_size=(2, 2)))

# Appiattisce i dati prima della classificazione finale
model.add(layers.Flatten())

# Layer di Dropout (spegne casualmente alcuni neuroni per evi
tare l'overfitting)
model.add(layers.Dropout(0.5))

# Output: Un nodo per ogni classe in TARGET_LABELS con attiva
zione softmax
model.add(layers.Dense(NUM_CLASSES, activation="softmax"))

model.summary()
```

Inizialmente non è che fosse proprio buono il riconoscimento, come si può notare dalla foto "primoris.png". La colpa però non era tutto del modello, ma del dataset che abbiamo usato per il training. Questo dataset (EMNIST) ha varie caratteristiche ben precise:

- le lettere riempiono quasi interamente tutto lo spazio disponibile nella foto;
- sono scalate per occupare un quadrato di 20×20 pixel, esattamente al centro del riquadro totale di 28×28 pixel.

Questo è quindi il problema: per la rete neurale, quella piccola concentrazione di pixel quasi verticale in mezzo al quadrato assomiglia statisticamente di più al pattern di un "1" piuttosto che alle ampie curve di una "S" (che il modello si aspetta arrivino quasi fino ai bordi).

Per questo motivo, abbiamo aggiunto la riga di codice

```
model.add(layers.Dense(128, activation="relu"))
```

che fornisce un layer intermedio per fornire più capacità di "ragionamento" alla rete (ancora da spiegare come).

Potenziamento della generalizzazione

Dopo la prima correzione, abbiamo riscontrato una precisione di 3 previsioni corrette e 4 errate su esempi scritti a mano da noi. La situazione è già migliorata con l'aggiunta del layer per il ragionamento, ma non riesce a generalizzare a sufficienza.

Abbiamo quindi aggiunto un blocco di Data Augmentation, con la funzionalità di ruotare, zoommare e traslare leggermente le immagini del training set prima di fornirle alla rete.

```
# Applichiamo distorsioni casuali ma leggere, per non trasfor  
mare un '6' in un '9' capovolto!  
# Rotazione casuale fino a ~18 gradi  
model.add(layers.RandomRotation(factor=0.05))  
# Zoom in/out fino al 10%  
model.add(layers.RandomZoom(height_factor=0.1, width_factor=
```

```
0.1))  
# Spostamento laterale/verticale del 10%  
model.add(layers.RandomTranslation(height_factor=0.1, width_factor=0.1))
```

Purtroppo però, questa modifica non ha portato ad alcun miglioramento nello score del ML, e abbiamo quindi deciso di annullare questa modifica e ripristinare lo stato di allenamento a quello precedente.

Modifica delle immagini di inferenza

Abbiamo quindi pensato a smussare un po' le foto dei caratteri che vengono fornite al modello in fase di inferenza, in modo da renderle più "sfuocate" e quindi simili a quelle contenute nel training set. Abbiamo quindi aggiunto un filtro di sfocatura:

```
# Ammorbidiamo il tratto per renderlo simile a inchiostro vero  
img = img.filter(ImageFilter.GaussianBlur(radius=0.8))
```

Ovviamente questa modifica non è stata fatta nello script di allenamento del ML, ma in quello che utilizziamo nella fase di inferenza del modello. Anche questo tentativo non ha portato a grandi miglioramenti, abbiamo quindi deciso di scartare anche questa.

Modifica delle immagini di inferenza 2

A questo punto abbiamo, sempre modificando il codice per la parte di inferenza, deciso di modificare molto di più rispetto che a una semplice sfuocatura: abbiamo deciso di processare le immagini delle lettere esattamente nello stesso modo in cui venivano processate nel training set:

- ritaglio delle parti della foto inutili (che non contengono inchiostro sostanzialmente);
- centratura della lettera al centro della foto;

- scalatura della lettera per renderla uguale, in pixel, a quelle del training set. (è stato sostituito interamente il codice, quindi non c'è una sola modifica vera e propria).

Si è rivelata la scelta giusta, portando a casa un punteggio di 6 caratteri predetti correttamente e 1 errato. Abbiamo notato però che la rete non era molto robusta al rumore posto vicino al carattere, infatti, come mostrato nella foto "seienove.png", possiamo notare come il puntino in alto a sinistra abbia confuso il ML facendogli credere la lettera iniziasse molto più in alto di dove inizia realmente. Di conseguenza, il "9" è stato schiacciato verso il basso, decentrato, e la CNN, vedendo un tondo compresso in basso con una linea sopra (il pixel isolato), ha "visto" un "6" un po' sbilenco.

Filtro anti-rumore

Fino ad ora, consideravamo parte dei caratteri qualsiasi pixel che non fosse nero (colore dello sfondo) assoluto. Abbiamo quindi cambiato il codice introducendo una ricerca intelligente, per capire quali tratti facciano effettivamente parte di un carattere e quali siano solamente rumore. Abbiamo quindi sostituito questa parte di codice nella funzione `centra_e_ridimensiona`:

```
# 1. Trova le coordinate dell'inchiostro (pixel non neri)
righe_con_pixel = np.any(arr > 10, axis=1)
colonne_con_pixel = np.any(arr > 10, axis=0)
```

con questa:

```
# 1. Trova le coordinate dell'inchiostro ignorando il "rumor
e" o i pixel sbiaditi
# Aumentando la soglia a 50 (invece di 10), ignoriamo i grigi
molto scuri.
# Inoltre, chiediamo che ci siano ALMENO 2 pixel vicini per c
onsiderare valida la riga/colonna
righe_con_pixel = np.sum(arr > 50, axis=1) > 2
colonne_con_pixel = np.sum(arr > 50, axis=0) > 2
```

Abbiamo notato però che l'errore persisteva sulla stessa immagine, abbiamo quindi deciso di adottare un approccio più "severo".

Isole di pixel con SciPy

Siccome avevamo già installato la libreria scikit-learn, abbiamo deciso di usare la libreria **SciPy** per rilevare l'isola di pixel più grande e rimuovere tutte le restanti, andando difatti a eliminare completamente i puntini di rumore presenti nell'immagine. (la funzione `centra_e_ridimensiona` è stata quindi sostituita completamente con quella presente ora nel codice).

Ciò ha risolto il problema, portando il ML a una precisione del 100%.

Script per la digitalizzazione della mappa

Creazione del programma e prima prova

Data la natura del problema, ovvero una mappa disegnata a mano di confini contenente solo lettere e numeri, abbiamo optato per l'utilizzo di tecniche di pulizia immagine al posto di allenare un ML per la digitalizzazione della mappa. Abbiamo optato per la libreria **OpenCV**, anche perché avevamo già tutte le librerie necessarie installate.

L'idea iniziale era quella di:

- trasformare la foto in scala di grigi per rimuovere qualsiasi tipo di colore;
- rimpicciolimento della foto a una griglia 100×100 (anche se poi è diventato variabile, ma mantenendo le dimensioni di un quadrato);
- utilizzo di una Threshold per decidere se considerare il pixel un 1 (quindi nero, confini) o uno 0 (quindi bianco, sfondo).

Come si può notare dalla foto "discrV1.png" però, il programma aveva molti problemi con foto scattate su fogli a quadretti e senza flash.

Programma avanzato

Per risolvere questo problema, abbiamo deciso di usare una **Soglia Adattiva**, in modo da usare una threshold diversa in base alla zona della foto. In questo modo

viene calcolata una soglia diversa per ogni piccola zona (20×20), rendendo il programma "intelligente", in grado di distinguere sfondi con intensità diverse presenti nella stessa foto, e tratti con intensità diverse. Abbiamo inoltre applicato un **Median Blur**, un filtro in grado di cancellare linee sottili (come i quadretti dei fogli), ma mantenendo intatti e nitidi i tratti spessi (come un pennarello). (per la soluzione è stato creato un nuovo file: `DiscConfAvanz.py`).

Il primo risultato è mostrato nella foto "discrV1Avanz.png", e si può notare come ci sia un netto miglioramento rispetto alla versione precedente, ma non siamo ancora arrivati a una versione accettabile. Il problema è che, com'è possibile notare nella foto, non abbiamo usato un pennarello, ma una penna, rendendo il tratto molto più sottile. Questo comportamento è poi accentuato dalla riduzione di risoluzione della foto, in quanto la griglia risultante è 100×100.

Ispessimento del tratto

Per risolvere questo problema, abbiamo deciso di dilatare i tratti che consideriamo i confini, andando così ad eliminare il problema dei tratteggi causati dalla risoluzione. Abbiamo quindi aggiunto questa parte di codice nel blocco della rimozione delle ombre:

```
# Creiamo un "pennello" di 3x3 pixel
a2bbbc49-ed3f-47bd-bc09-64c947a00d21kernel = np.ones((3,3), n
p.uint8)
# Spalmiamo l'inchiostro rilevato per unire i buchi della pen
na
img_binaria = cv2.dilate(img_binaria, kernel, iterations=1)
```

In questo modo, prima prima che l'immagine venga pulita e ridimensionata, OpenCV prenderà ogni frammento del tratto di penna e lo farà "lievitare" di 1 pixel in tutte le direzioni, chiudendo i buchi creati dal contrasto debole e garantendo un confine continuo. Come possiamo vedere dalla foto "discrV2Avanz.png", il tratto del confine risulta quasi perfetto, a discapito dell'aumento del rumore.

Erosione Morfologica

Poiché abbiamo abbassato la soglia e il `medianBlur` per salvare il tratto debole della penna, i quadretti sono sopravvissuti al filtro. Essendo tutti collegati fisicamente alla tua linea, hanno formato **un'unica gigantesca "isola"**, rendendo inutile la funzione di pulizia finale. Per ottimizzare questo sistema senza perdere i tratti deboli della penna, abbiamo deciso di **ispessire la penna prima di sfocare l'immagine**. Per questo abbiamo usato un'operazione chiamata **Erosione Morfologica** presente in OpenCV, in grado di espandere i pixel scuri, rendendo il tratto più marcato e forte. Possiamo così quindi rialzare il filtro per rimuovere i quadretti senza perdere il disegno di interesse.

Abbiamo quindi riportato i parametri a livelli più alti:

```
medianBlur = 11 # Torniamo a 11 per rimuovere i quadretti
adaptiveThreshold = 10 # Torniamo a 10 per ignorare le ombre deboli
```

e aggiunto l'Erosione prima di applicare il filtro:

```
# Creiamo un pennello 2x2
kernel_penna = np.ones((2, 2), np.uint8)
# L'erosione in scala di grigi rende i tratti scuri più spessi!
img = cv2.erode(img, kernel_penna, iterations=1)
```

Dalla foto "discrV3Avanz.png" si nota immediatamente come siamo riusciti a rimuovere il rumore e a migliorare il confine del disegno.

Chiusura dei buchi

A questo punto possiamo usare l'operazione di **Chiusura Morfologica** per chiudere perfettamente i buchi, e risolvere finalmente il problema dei confini.

Abbiamo quindi sostituito la riga

```
# Spalmiamo l'inchiostro rilevato per unire i buchi della penna
img_binaria = cv2.dilate(img_binaria, kernel, iterations=1)
```


con queste

```
# Creiamo un "pennello" leggermente più grande (5x5)
kernel_chiusura = np.ones((5, 5), np.uint8)
# MORPH_CLOSE salda i buchi interni al tratto senza ingrassare
# e troppo la linea
img_binaria = cv2.morphologyEx(img_binaria, cv2.MORPH_CLOSE,
kernel_chiusura, iterations=2)
```

Abbiamo quindi aumentato la dimensione del pennello a un 5×5 ripetendola due volte. In questo modo, il modello per la discretizzazione è stato completato, e andava solo integrato con altre piccole modifiche al ML per il riconoscimento dei caratteri.

Il risultato finale è mostrato nella foto "discrFinaleAvanz.png".

Modello Completo

A questo punto, dobbiamo preparare un programma completo in grado di passare dalla foto della mappa, con le lettere scritte dentro gli stati, alla versione digitalizzata, con colori diversi per stati diversi, riconoscimento e conseguente rimozione delle lettere dalla mappa. Abbiamo quindi creato tre file differenti:

- `modulo_digitalizzatore.py`: deve preparare l'immagine, ispessirla l'inchiostro e rimuovere gli eventuali quadretti, restituendo la mappa con tutto l'inchiostro trovato;
- `modulo_ml.py`: ovvero l'utilizzatore della rete neurale, che prenderà il ritaglio del carattere pulito, lo scalerà a un 28×28 e restituirà il carattere predetto;
- `hub_master.py`: ovvero il controllore generale. Dovrà analizzare l'area delle "isole" per separarle, inviare le lettere al `modulo_ml`, colorare le isole grandi (gli stati) e impacchettare tutto nella matrice risultante.

Spiegazione `modulo_digitalizzatore.py`

Spiegazione `modulo_ml.py`

Spiegazione `hub_master.py`

Primi risultati

Il primo risultato è mostrato nella foto "completoV1.jpeg". Come si può notare c'è un'esplosione di falsi positivi, e questo è dovuto al fatto che in questo modello, quello completo, manca la funzione che rimuove tutto il rumore una volta trovata l'isola di pixel principale, che invece nelle versioni singole precedenti esisteva. Questo permette ai numeri scritti all'interno degli stati di sopravvivere alla rimozione del rumore, per poter essere poi predetti dal ML.

La soglia impostata di partenza per rilevare un carattere come tale era troppo piccola, e questo non solo spiega perché quelli rilevati sono scritti molto piccoli, ma spiega anche perché quelli reali non sono stati predetti: perché sono troppo grandi.

Correzioni finali

Per risolvere abbiamo quindi ottimizzato dei parametri, come la dimensione minima (300) per considerare un'isola di pixel un carattere e la dimensione massima consentita (15000).

Come si può vedere infatti dalla foto "completoV2.jpeg", questa modifica ha portato i suoi frutti, riuscendo a indentificare tutte i caratteri correttamente. In realtà, compare anche un "7" di troppo, trovato sicuramente nei pixel di rumore. Questo però non pone alcun problema al risultato finale, in quanto essendo "fuori" dagli stati, non verrà considerato nel calcolo del percorso (questa sicurezza è data dalla funzione `pointPolygonTest`, che sa che solo i caratteri contenuti dentro un confine chiuso hanno un valore (non so comunque perché)).

Correzione dei colori

Come si nota sempre dalla foto precedente, tutti gli stati sono colorati con gli stessi colori. Impostiamo quindi un sistema che colori per intero gli stati, ognuno con un colore diverso dallo stato adiacente. Non useremo però un algoritmo complesso come il classico "Teorema dei quattro colori", ma assegneremo semplicemente un colore unico a ogni territorio. Abbiamo creato quindi una palette di 12 colori e modificato un'istruzione di OpenCV: `cv2.RETR_EXTERNAL`, che in questo modo cerca solamente il contorno esterno ignorando quelli interni.

Useremo quindi `connectedComponents`, che permetterà al programma di cercare i vari stati diversi, utilizzando le linee nere come ostacoli. In questo modo, come possiamo notare dal risultato finale "Risultato.jpeg", la divisione degli stati è perfetta.